

Assignment: NET Centric Computing

Course Code: CSC378

Topic: Unit 4 - Creating ASP.NET Core MVC Applications (Theory & Practical)

Prepared by: Mr. Subash Pariyar

Due Date: 24 August

Submission Instructions:

Please complete the following questions by combining theoretical explanations with practical implementations. For the practical components, include required commands, code snippets, project structure details, and screenshots of application execution and testing results.

Questions:

1. Developing an ASP.NET Core MVC Application with Controllers, Actions, Views, and Model Validation

Create an ASP.NET Core MVC application to demonstrate controllers, actions, action results, Razor views, model binding, and server-side validation. Explain the role of controllers, actions, action result types, Razor views, and model validation in ASP.NET Core MVC applications.

Tasks:

- Create a new ASP.NET Core MVC project.
- Create controllers and implement actions.
- Implement Action Result types including ViewResult, JsonResult, RedirectResult, ContentResult, and NotFoundResult.
- Create models and apply Data Annotation validation.
- Implement model binding for form data.
- Create Razor Views using Razor syntax and Tag Helpers.
- Implement CRUD operations.
- Configure URL routing.
- Document controllers, actions, views, validation, routing, and screenshots.

2. Developing ASP.NET Core Web API with JSON, Dependency Injection, and IoC Container

Develop an ASP.NET Core Web API application to demonstrate API controllers, JSON data exchange, Dependency Injection, and IoC container concepts.

Tasks:

- Create an ASP.NET Core Web API project.
- Create a Product API Controller with:
ProductId, Name, Price, and Category.
- Implement GET, POST, PUT, and DELETE operations.
- Configure JSON responses.
- Test APIs using Swagger or Postman.
- Analyze request format, response format, JSON structure, and HTTP status codes.
- Implement Dependency Injection using interface and service classes.
- Register services in IoC container.
- Demonstrate Transient, Scoped, and Singleton service lifetimes.
- Document API structure, JSON examples, DI workflow, IoC configuration, and screenshots.